

Spring Boot Filters - 2

Modifying Requests and Responses, Filter Registration, and `OncePerRequestFilter`

A servlet Filter can do more than inspect an HTTP request and response. It can also change what is sent to the client or what downstream components receive.

However, request and response modification do not work in the same way:

- Response headers and status codes are relatively easy to change.
- Response bodies usually require a response wrapper.
- Request headers and parameters are read-only from the original request object.
- Request-body handling is more sensitive because the body arrives as a stream.

Request and response mutation is possible, but it should be used only when the Filter layer is genuinely the correct place for that responsibility.

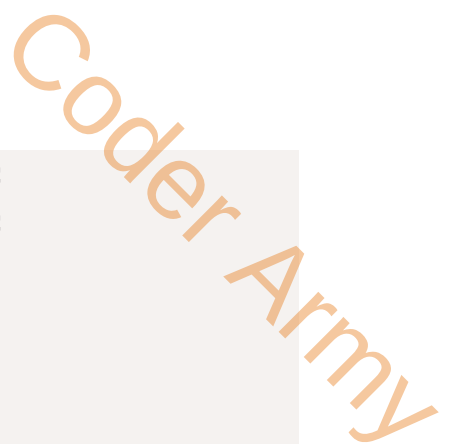
1. Modifying the HTTP Response

1.1 Adding a response header

A Filter can add or replace response headers before passing the request further through the chain.

```
package in.strikes.crudDemo.filter;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
```



```
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Component;

import java.io.IOException;

@Component
public class ResponseHeaderFilter implements Filter {

    @Override
    public void doFilter(
        ServletRequest request,
        ServletResponse response,
        FilterChain chain
    ) throws IOException, ServletException {

        HttpServletResponse httpResponse =
            (HttpServletResponse) response;

        httpResponse.setHeader(
            "X-Application-Name",
            "Student Management API"
        );

        chain.doFilter(request, response);
    }
}
```

The client now receives a header similar to:

```
X-Application-Name: Student Management API
```

setHeader() vs **addHeader()**

```
httpResponse.setHeader("X-Application-Name", "Student API");
```

`setHeader()` replaces the existing value of that header.

```
httpResponse.addHeader("X-Feature", "Students");  
httpResponse.addHeader("X-Feature", "Courses");
```

`addHeader()` can add another value without replacing the previous one.

1.2 Why headers are usually added before `chain.doFilter()`

```
httpResponse.setHeader(  
    "X-Application-Name",  
    "Student Management API"  
);  
  
chain.doFilter(request, response);
```

The downstream code may write and flush the response. Once that happens, the response can become **committed**.

A committed response means that its status and headers may already have been sent to the client. After that point, changing them may have no effect or may cause an error.

The response state can be checked using:

```
boolean committed = httpResponse.isCommitted();
```

Add headers early when they should be present on every response. Logic that depends on the final result may need to run after `chain.doFilter()` and must account for the response possibly being committed.

2. Setting the Response Status

A Filter can assign an HTTP status code.

```
httpResponse.setStatus(  
    HttpServletResponse.SC_UNAUTHORIZED  
);
```

The constant above represents HTTP status `401 Unauthorized`.

A Filter can also stop request processing and return a complete response directly.

```
httpResponse.setStatus(  
    HttpServletResponse.SC_UNAUTHORIZED  
);  
  
httpResponse.setContentType("application/json");  
httpResponse.setCharacterEncoding("UTF-8");  
  
httpResponse.getWriter().write(  
    "{  
      \"message\": \"Authentication required\"  
    }"  
);  
  
return;
```

Because the method returns without calling `chain.doFilter()`, the request does not reach the next Filter or the controller.

Important rule

```
chain.doFilter(request, response);
```

means:

| Continue processing the request.

```
return;
```

without calling the chain means:

| Stop processing the request here.

Practical note

For production authentication and authorization, Spring Security is generally the correct solution. For example, authentication failures are commonly handled through components such as `AuthenticationEntryPoint`, rather than a general homemade Filter.

3. Modifying the Response Body

Suppose a controller returns:

```
{
  "name": "Aditya",
  "age": 25
}
```

The desired response is:

```
{
  "data": {
    "name": "Aditya",
    "age": 25
  },
  "application": "Student Management API"
}
```

A normal `HttpServletResponse` does not provide a method such as:

```
String body = response.getBody(); // Does not exist
```

Why is there no `getBody()` method?

The controller or Spring MVC writes response data to:

```
response.getWriter();
```

or:

```
response.getOutputStream();
```

The data may then move progressively toward the client. The ordinary response acts mainly as a destination into which data is written. It is not designed as a Java object from which the complete body can later be retrieved.

```
Controller
  ↓ writes
HttpServletResponse output stream
  ↓
Client
```

To examine or replace the body after downstream processing, the response must first be wrapped.

4. Using `ContentCachingResponseWrapper`

Spring provides:

```
ContentCachingResponseWrapper
```

This wrapper stores a copy of the response body so that it can be accessed after the controller has executed.

Conceptual flow

```
Client request
  ↓
ResponseBodyFilter
```

```

    ↓ creates wrapper
ContentCachingResponseWrapper
    ↓ passed through chain
Controller writes response into wrapper
    ↓
Filter reads cached response bytes
    ↓
Filter optionally replaces the body
    ↓
Final body is copied to real response
    ↓
Client receives response

```

Example

```

package in.strikes.crudDemo.filter;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.util.ContentCachingResponseWrapper;

import java.io.IOException;
import java.nio.charset.StandardCharsets;

@Component
public class ResponseBodyFilter implements Filter {

    @Override
    public void doFilter(
        ServletRequest request,
        ServletResponse response,

```

```

        FilterChain chain
    ) throws IOException, ServletException {

        HttpServletRequest httpRequest =
            (HttpServletRequest) request;

        HttpServletResponse httpResponse =
            (HttpServletResponse) response;

        ContentCachingResponseWrapper wrappedResponse =
            new ContentCachingResponseWrapper(httpRespons
e);

        // Downstream code writes into the wrapper.
        chain.doFilter(httpRequest, wrappedResponse);

        byte[] originalBodyBytes =
            wrappedResponse.getContentAsByteArray();

        String originalBody = new String(
            originalBodyBytes,
            StandardCharsets.UTF_8
        );

        String modifiedBody =
            """
            {
                "originalResponse": %s,
                "processedBy": "ResponseBodyFilter"
            }
            """.formatted(originalBody);

        byte[] modifiedBodyBytes =
            modifiedBody.getBytes(StandardCharsets.UTF_
8);
    }

```



```

        // Clears the currently cached body, but keeps status
        and headers.
        wrappedResponse.resetBuffer();

        wrappedResponse.setContentType("application/json");
        wrappedResponse.setCharacterEncoding(
            StandardCharsets.UTF_8.name()
        );

        wrappedResponse.getOutputStream()
            .write(modifiedBodyBytes);

        // Copies the cached final body into the actual response.
        wrappedResponse.copyBodyToResponse();
    }
}

```

Important steps

Step 1: Wrap the real response

```

ContentCachingResponseWrapper wrappedResponse =
    new ContentCachingResponseWrapper(httpResponse);

```

Step 2: Pass the wrapper through the chain

```

chain.doFilter(httpRequest, wrappedResponse);

```

The controller now writes into `wrappedResponse`, not directly into the original response.

Step 3: Read the cached body

```
byte[] originalBodyBytes =  
    wrappedResponse.getContentAsByteArray();
```

Step 4: Clear the cached body before replacing it

```
wrappedResponse.resetBuffer();
```

`resetBuffer()` clears the body buffer while retaining the existing status code and headers.

Step 5: Write the replacement body

```
wrappedResponse.getOutputStream()  
    .write(modifiedBodyBytes);
```

Step 6: Copy the body to the real response

```
wrappedResponse.copyBodyToResponse();
```

This step is essential. Without it, the client may receive an empty response because the data remains inside the wrapper.

4.1 Limitations of manually modifying a response body

The example is useful for understanding wrappers, but manually rebuilding JSON inside a Filter is uncommon in normal application code.

Potential problems include:

- The original body may be empty.
- The body may not contain JSON.
- The original JSON may be malformed.
- Binary responses such as files or images must not be treated as text.
- Buffering large responses consumes memory.

- Existing headers such as `Content-Length`, `ETag`, or compression-related headers may become invalid after modification.
- Streaming responses are not suitable for complete-body caching.
- Exceptions and error dispatches require careful handling.

This expression:

```
""  
{  
  "originalResponse": %s  
}  
"".formatted(originalBody);
```

assumes that `originalBody` is already valid JSON.

For production JSON transformation, use Jackson rather than constructing JSON manually.

For globally changing controller response bodies in Spring MVC, `ResponseBodyAdvice` is often a cleaner choice than a servlet Filter.

5. Modifying Request Data

Request modification is generally more complex than response-header modification.

Suppose the client sends:

```
GET /api/students  
Authorization: Bearer abc123
```

The Filter can read the header:

```
String token =  
    httpRequest.getHeader("Authorization");
```

However, the original request does not provide a normal method such as:

```
httpRequest.setHeader(  
    "Authorization",  
    "Bearer new-token"  
); // Does not exist
```

`HttpServletRequest` is primarily a read-oriented interface. To change what downstream code sees, the normal technique is to pass a wrapper instead of the original request.

6. The Request Wrapper Concept

```
Original HttpServletRequest  
    ↓ wrapped by  
Custom HttpServletRequestWrapper  
    ↓ passed to  
Next Filter or DispatcherServlet
```

A custom wrapper can override methods such as:

```
getHeader(String name)  
getHeaders(String name)  
getHeaderNames()  
getParameter(String name)  
getParameterMap()  
getParameterValues(String name)  
getInputStream()  
getReader()
```

Downstream components receive the wrapper and therefore observe the values returned by its overridden methods.

Base class

```
HttpServletRequestWrapper
```

A custom request wrapper normally extends this class:

```
public class CustomRequestWrapper
    extends HttpServletRequestWrapper {

    public CustomRequestWrapper(
        HttpServletRequest request
    ) {
        super(request);
    }
}
```

The wrapper delegates unchanged operations to the original request and overrides only the behaviour that needs to change.

Request mutation through wrappers is valid, but it should be used carefully. Business-level transformations are often better placed in Spring MVC, validation, Jackson configuration, or the service layer.

7. Modifying Query Parameters

Query parameters are also read-only from the perspective of the original request. Suppose the request is:

```
GET /products?page=1
```

This method does not exist:

```
request.setParameter("page", "2");
```

To change how downstream code reads parameters, a wrapper must consistently override methods such as:

```
getParameter(String name)
getParameterMap()
getParameterValues(String name)
getParameterNames()
```

Overriding only `getParameter()` may be insufficient because Spring MVC or another library may read the complete parameter map instead.

Use this technique carefully because the same parameters may be consumed by:

- `@RequestParam`
- Data binding
- Validation
- Other Filters
- Spring MVC infrastructure

8. Understanding the Request Body

Request-body handling is more sensitive than reading headers or query parameters.

Common request information is exposed through methods such as:

```
httpRequest.getHeader("Authorization");
httpRequest.getHeaderNames();
httpRequest.getParameter("page");
httpRequest.getParameterMap();
httpRequest.getMethod();
httpRequest.getRequestURI();
```

However, this method does not exist:

```
httpRequest.getBody(); // Does not exist
```

The request body is accessed as a stream:

```
request.getInputStream();
```

or as a character reader:

```
request.getReader();
```

8.1 Why is the request body exposed as a stream?

The body can contain many different kinds of data:

- JSON
- XML
- Plain text
- PDF documents
- Images
- Videos
- Multipart form data
- Other binary content

The servlet container cannot assume:

- That the body contains text
- That it contains JSON
- Which Java class it should become
- How large it is
- Which parser should process it
- Which character encoding should be used

At the HTTP level, the body is fundamentally a sequence of bytes arriving from the client.

```

Client sends bytes
    ↓
ServletInputStream
    ↓
Spring MVC selects an HttpMessageConverter
    ↓
Jackson handles JSON when applicable
    ↓
Java object
  
```

For a JSON request handled with `@RequestBody`, Jackson commonly converts those bytes into the required Java object.

```

@PostMapping
public Student createStudent(
    @RequestBody Student student
) {
    return student;
}
  
```

9. Why the Request Body Normally Cannot Be Read Repeatedly

An input stream has a current position. Each read moves that position forward.

Suppose the client sends:

```

{
  "name": "Aditya"
}
  
```

The server receives a sequence of bytes:

```

{ " n a m e " : ...
  
```

The read position moves through the stream:

Start
↓
{ → " → n → a → m → e → " → : → ...
↓
End of stream

After the bytes have been consumed, the stream is at its end. The original network bytes are not automatically placed back at position zero.

9.1 Incorrectly reading the body inside a Filter

Controller:

```
@RestController
@RequestMapping("/api/students")
public class StudentController {

    @PostMapping
    public Student createStudent(
        @RequestBody Student student
    ) {

        System.out.println(
            "Controller received: " + student
        );

        return student;
    }
}
```

Filter:

```
@Component
public class RequestBodyReadingFilter
    implements Filter {
```

```
@Override
public void doFilter(
    HttpServletRequest request,
    HttpServletResponse response,
    FilterChain chain
) throws IOException, ServletException {

    HttpServletRequest httpRequest =
        (HttpServletRequest) request;

    BufferedReader reader =
        httpRequest.getReader();

    StringBuilder body =
        new StringBuilder();

    String line;

    while ((line = reader.readLine()) != null) {
        body.append(line);
    }

    System.out.println(
        "Filter read body: " + body
    );

    chain.doFilter(request, response);
}
}
```

The Filter may print the body successfully. However, when Spring MVC later tries to create the `Student` object, the stream may already be empty.

The controller can then fail with an error similar to:

Required request body is missing

9.2 Reading through `InputStream`

The same issue occurs when using `getInputStream()`:

```
InputStream inputStream =
    request.getInputStream();

String body = new String(
    inputStream.readAllBytes(),
    StandardCharsets.UTF_8
);

System.out.println(body);
```

`getReader()` and `getInputStream()` are two different views of the same request body. Reading the body through either one consumes it.

The Servlet API also does not allow an application to freely mix both methods on the same request.

10. Why Java and Tomcat Do Not Automatically Rewind Every Body

Automatic repeated reading would require the server to buffer the complete body somewhere.

Consider a request containing a 2 GB video:

```
Receive 2 GB
  ↓
Store 2 GB in memory or on disk
  ↓
Keep it available for possible repeated reads
```

Doing this automatically for every request would:

- Consume large amounts of memory or disk space
- Increase latency
- Reduce throughput
- Create denial-of-service risks
- Be unnecessary for most requests

Streaming therefore allows the server to process data progressively without storing every request body in full.

11. Re-reading or Replacing a Request Body

A Filter that must read and still preserve the body needs a request wrapper.

Typical use cases include:

- Logging selected request data
- Masking sensitive fields before logging
- Decrypting an encrypted payload
- Sanitising input
- Replacing or transforming request content

The general strategy is:

```

Read the original request stream once
    ↓
Store the bytes in a byte array
    ↓
Pass a custom HttpServletRequestWrapper
    ↓
Wrapper returns a fresh stream over cached bytes
    ↓
Spring MVC can read the body normally
  
```

A wrapper commonly overrides both:

```
getInputStream()  
getReader()
```

Whenever downstream code calls `getInputStream()`, the wrapper can create a new stream over the stored bytes.

```
new ByteArrayInputStream(cachedBody);
```

A new `ByteArrayInputStream` begins at position zero, allowing the cached body to be read again.

Memory warning

Caching a request body means storing it in memory unless a more advanced disk-backed strategy is used.

Before buffering, consider:

- Maximum request size
- File uploads
- Multipart requests
- Sensitive information
- Passwords and tokens
- Performance impact

Never log complete request bodies blindly. Logs can expose credentials, personal data, access tokens, and financial information.

12. Choosing the Correct Layer

Before modifying data inside a Filter, consider whether another Spring mechanism is more suitable.

Requirement	Usually better suited mechanism
Authentication and authorization	Spring Security
Standard controller response envelope	<code>ResponseBodyAdvice</code>
Exception response formatting	<code>@RestControllerAdvice</code>
JSON field conversion	Jackson configuration or custom deserializer
Controller argument creation	<code>HandlerMethodArgumentResolver</code>
Input validation	Bean Validation
Business-data transformation	Service layer
Low-level HTTP concern	Filter

Filters are most appropriate for concerns that truly belong at the servlet or HTTP boundary.

13. Registering Filters in Spring Boot

There are multiple ways to register and scope a Filter.

Approach 1: Register with `@Component`

```
@Component
public class RequestLoggingFilter implements Filter {
    // Filter logic
}
```

Spring Boot detects the bean and registers it with the servlet container.

This approach is simple, but the Filter is generally mapped broadly, commonly to `/*`.

Use it when:

- The Filter should apply broadly
- Advanced registration options are unnecessary
- The registration should remain simple

14. Registering with `FilterRegistrationBean`

`FilterRegistrationBean` provides explicit control over Filter registration.

It can configure:

- URL patterns
- Filter order
- Filter name
- Dispatcher types
- Whether registration is enabled
- Async support

Why use it?

Suppose a Filter should apply only to:

```
/api/*
```

and not to:

```
/actuator/*
/swagger-ui/*
/static/*
```

`FilterRegistrationBean` can restrict the Filter at registration time.

14.1 Step 1: Create a normal Filter class

Do not annotate it with `@Component` when the same instance will be registered manually through `FilterRegistrationBean`.

```
package in.strikes.crudDemo.filter;
```

```
import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.http.HttpServletRequest;

import java.io.IOException;

public class RequestLoggingFilter implements Filter {

    @Override
    public void doFilter(
        ServletRequest request,
        ServletResponse response,
        FilterChain chain
    ) throws IOException, ServletException {

        HttpServletRequest httpRequest =
            (HttpServletRequest) request;

        System.out.println(
            "Filtered request: "
            + httpRequest.getRequestURI()
        );

        chain.doFilter(request, response);
    }
}
```

14.2 Step 2: Register it in a configuration class

```
package in.strikes.crudDemo.config;
```



```

import in.strikes.crudDemo.filter.RequestLoggingFilter;
import org.springframework.boot.web.servlet.FilterRegistration
nBean;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class FilterConfig {

    @Bean
    public FilterRegistrationBean<RequestLoggingFilter>
        requestLoggingFilterRegistration() {

        FilterRegistrationBean<RequestLoggingFilter>
            registrationBean =
                new FilterRegistrationBean<>();

        registrationBean.setFilter(
            new RequestLoggingFilter()
        );

        registrationBean.addUrlPatterns("/api/*");
        registrationBean.setOrder(1);
        registrationBean.setName(
            "requestLoggingFilter"
        );

        return registrationBean;
    }
}

```

The Filter now applies to paths such as:

```

/api/students
/api/products

```

```
/api/orders/10
```

It does not apply to paths such as:

```
/hello  
/actuator/health  
/swagger-ui/index.html
```

unless another configured pattern matches them.

14.3 Registering multiple URL patterns

```
registrationBean.addUrlPatterns(  
    "/api/*",  
    "/admin/*"  
);
```

The Filter will apply to both groups:

```
/api/students  
/admin/users
```

14.4 Avoid accidental duplicate registration

Do not normally use both of these for the same Filter:

```
@Component  
public class RequestLoggingFilter implements Filter {  
}
```

and:

```
registrationBean.setFilter(  
    new RequestLoggingFilter()
```

```
);
```

Doing both can create two Filter registrations and cause the logic to execute more than once.

Choose one registration strategy unless duplicate registrations are intentional.

15. Controlling Filter Order

When multiple Filters are registered, their order determines how the request and response pass through them.

```
registrationBean.setOrder(1);
```

A lower order value runs earlier while processing the request.

Suppose:

```
LoggingFilter order = 1  
AuthenticationFilter order = 2
```

Request flow:

```
Client  
↓  
LoggingFilter  
↓  
AuthenticationFilter  
↓  
DispatcherServlet  
↓  
Controller
```

Response flow returns in reverse:

```
Controller  
↓  
AuthenticationFilter  
↓
```

```
LoggingFilter
↓
Client
```

This nested behaviour occurs because each Filter executes code before and after `chain.doFilter()`.

16. Filtering Specific URLs Manually

A Filter can inspect the current URI and decide whether to run its custom logic.

```
@Component
public class ApiFilter implements Filter {

    @Override
    public void doFilter(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain chain
    ) throws IOException, ServletException {

        HttpServletRequest httpRequest =
            (HttpServletRequest) request;

        String uri =
            httpRequest.getRequestURI();

        if (!uri.startsWith("/api/")) {
            chain.doFilter(request, response);
            return;
        }

        System.out.println(
            "API-specific filter logic"
        );
    }
}
```

```
        chain.doFilter(request, response);
    }
}
```

This works, but the Filter is still invoked for every URL included in its servlet registration. It then decides internally whether to perform the custom logic.

Use this approach when:

- The condition is small and easy to understand
- The decision depends on more than a simple URL pattern
- The Filter is already registered broadly

For clean URL-level registration, `FilterRegistrationBean` is generally clearer.

17. Using `OncePerRequestFilter`

Implementing the servlet `Filter` interface directly is completely valid.

For Spring web applications, Spring also provides:

`OncePerRequestFilter`

It is an abstract Spring class designed specifically for HTTP request filtering.

Example

```
package in.strikes.crudDemo.filter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
```

```
import java.io.IOException;

@Component
public class RequestLoggingFilter
    extends OncePerRequestFilter {

    private static final Logger logger =
        LoggerFactory.getLogger(
            RequestLoggingFilter.class
        );

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain
    ) throws ServletException, IOException {

        logger.info(
            "{} {}",
            request.getMethod(),
            request.getRequestURI()
        );

        filterChain.doFilter(request, response);
    }
}
```

Advantages

- Direct access to `HttpServletRequest`
- Direct access to `HttpServletResponse`
- No manual casting from `ServletRequest` and `ServletResponse`

- Designed for Spring-based web applications
- Prevents duplicate execution within the same dispatch using an internal request attribute
- Provides extension methods for controlling filtering behaviour

Important nuance

The name does not mean that the Filter can never run again for every possible dispatch associated with a request.

Servlet processing can involve dispatch types such as:

- `REQUEST`
- `ASYNC`
- `ERROR`
- `FORWARD`
- `INCLUDE`

`OncePerRequestFilter` provides once-per-dispatch protection and offers hooks for controlling async and error dispatch behaviour.

For most normal Spring MVC requests, it behaves as developers expect: the Filter logic runs once for the main request processing.

18. Excluding URLs with `shouldNotFilter()`

When extending `OncePerRequestFilter`, URL-exclusion logic can be placed in:

```
shouldNotFilter()
```

Example

```
package in.strikes.crudDemo.filter;

import jakarta.servlet.FilterChain;
```

```
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
public class ApiLoggingFilter
    extends OncePerRequestFilter {

    @Override
    protected boolean shouldNotFilter(
        HttpServletRequest request
    ) {

        String path = request.getRequestURI();

        return !path.startsWith("/api/");
    }

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain
    ) throws ServletException, IOException {

        System.out.println(
            "Filtering: "
            + request.getRequestURI()
        );

        filterChain.doFilter(request, response);
    }
}
```



```
}  
}
```

Understanding the return value

```
return true;
```

means:

Do not run `doFilterInternal()` for this request.

```
return false;
```

means:

Run `doFilterInternal()` for this request.

In the example:

```
return !path.startsWith("/api/");
```

- `/api/students` produces `false`, so the Filter runs.
- `/hello` produces `true`, so the Filter is skipped.

19. Comparing URL-Scoping Approaches

Approach	Where the decision occurs	Best suited for
<code>FilterRegistrationBean</code> URL patterns	Servlet registration	Clean, broad URL-pattern control
Manual URI check inside <code>doFilter()</code>	Inside Filter execution	Small or custom runtime conditions
<code>OncePerRequestFilter.shouldNotFilter()</code>	Spring Filter hook	Clean exclusions in a Spring-specific Filter

General guidance

Use `FilterRegistrationBean` when the Filter should not be registered for unrelated paths at all.

Use `shouldNotFilter()` when using `OncePerRequestFilter` and the inclusion or exclusion rule naturally belongs inside that Filter.

Use manual checks only when they remain simple or depend on custom request information.

20. `Filter` vs `OncePerRequestFilter`

Point	<code>Filter</code>	<code>OncePerRequestFilter</code>
API	Jakarta Servlet API	Spring Framework
Request type	<code>ServletRequest</code>	<code>HttpServletRequest</code>
Response type	<code>ServletResponse</code>	<code>HttpServletResponse</code>
Casting required	Usually yes	No
Main method	<code>doFilter()</code>	<code>doFilterInternal()</code>
Built-in skip hook	No dedicated Spring hook	<code>shouldNotFilter()</code>
Dispatch protection	Must manage manually if needed	Built-in once-per-dispatch mechanism
Portability outside Spring	Greater	Spring-specific

Both are valid. In a Spring Boot HTTP application, `OncePerRequestFilter` often produces cleaner code.

21. Practical Guidelines

Prefer a Filter for

- Low-level request and response logging
- Correlation or trace IDs

- Cross-origin handling when not using existing framework support
- Compression or encoding concerns
- Security-chain integration
- Request timing across the complete servlet flow
- HTTP-boundary concerns that apply before Spring MVC

Avoid using a Filter for

- Core business logic
- Database operations unrelated to the HTTP boundary
- Normal DTO mapping
- Controller-specific behaviour
- Complex response restructuring when `ResponseBodyAdvice` is sufficient
- Authentication systems that should use Spring Security
- Blind logging of passwords, tokens, or complete request bodies

22. Final Mental Model

Headers

- Easy to read
- Response headers are easy to add
- Request headers require a wrapper to appear modified

Query parameters

- Easy to read
- Require a wrapper to appear modified

Request body

- Arrives as a stream
- Normally consumed once
- Requires caching and a wrapper for repeated reading

Response body

- Written to an output stream
- Requires a response wrapper to inspect or replace later

Filter registration

- `@Component` for simple broad registration
- `FilterRegistrationBean` for explicit registration control
- `OncePerRequestFilter` for a Spring-oriented Filter base class

23. Key Takeaways

1. A Filter can stop a request by returning without calling `chain.doFilter()`.
2. Add important response headers before the response becomes committed.
3. A normal `HttpServletResponse` does not expose the completed body through `getBody()`.
4. `ContentCachingResponseWrapper` allows the response body to be inspected after downstream processing.
5. Always call `copyBodyToResponse()` when using `ContentCachingResponseWrapper` to preserve the final response.
6. The original `HttpServletRequest` does not provide setters for headers or query parameters.
7. Request wrappers change what downstream components see without changing the original request object.
8. Reading a request body in a Filter can consume the stream before the controller receives it.
9. Re-readable request bodies require caching and an overridden `getInputStream()` and `getReader()`.
10. Buffering request or response bodies has memory, performance, privacy, and security costs.
11. `FilterRegistrationBean` provides URL-pattern and ordering control.
12. Avoid combining `@Component` registration and manual registration for the same Filter unless duplicate execution is intentional.
13. `OncePerRequestFilter` is often more convenient for Spring Boot applications.
14. Use `shouldNotFilter()` for clean request exclusions in a `OncePerRequestFilter`.

15. Use the narrowest and most appropriate Spring layer for each concern.

Coder Army